



SQL

Consultas Aninhadas

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br

- Consultas Aninhadas
 - In
 - Any
 - All
 - Subconsultas
 - Otimizações



DML - Select

- Consultas aninhadas (subconsultas)
 - Forma alternativa de especificar consultas envolvendo relacionamento entre tabelas
- Otimização
 - Filtragens de prévias dos dados
 - Limita o número de atributos e tuplas a serem combinados em caso de junções
- Cláusulas
 - nome_atributo [**NOT**] **IN** (consulta SQL)
 - nome_atributo [< | > | = | != | <>] **ANY** (consulta SQL)
 - nome_atributo [< | > | = | != | <>] **ALL** (consulta SQL)



- In

- Permite comparações elemento-conjunto
 - Testa se um valor pertence ou não a um conjunto
- **SELECT** lista_atributos **FROM** tabela1

WHERE

atributo **[NOT] IN** (conjunto | consulta SQL)

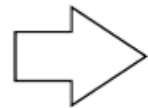
select $a_1, \dots, a_n$

from t_1

where c IN

(select x from t_2

where $d > 5$)



$\pi_{a_1, \dots, a_n} (t_1 \theta X (\pi_x (\sigma_{d > 5} (t_2))))$
 $\theta = t_1.c = t_2.x$

DML - In

Álgebra	SQL
$\pi_{\text{nome}} ($ $(\text{Médicos} \theta X$ $\theta = \text{Médicos.codm} = \text{Consultas.codm}$ $(\pi_{\text{codm}} (\sigma_{\text{data} = '13/11/2006'} (\text{Consultas})))$ $))$	Select nome From Médicos Where codm in (Select codm From Consultas Where data = '13/11/2006')
$(\pi_{\text{CPF}} (\text{Funcionários})) \text{ — } (\pi_{\text{CPF}} (\text{Pacientes}))$	Select CPF From Funcionários Where CPF not in (Select CPF From Pacientes)
$(\pi_{\text{CPF}} (\text{Médicos})) \cap (\pi_{\text{CPF}} (\text{Pacientes}))$	Select CPF From Médicos Where CPF in (Select CPF From Pacientes)



DML - Intersecção/Diferença

Intersecção/Diferença

Álgebra	SQL
$relação1 \text{ — } relação2$	<code>consultaSQL1 except consultaSQL2</code>
$relação1 \cap relação2$	<code>consultaSQL1 intersect consultaSQL2</code>

Exemplo

$(\pi_{CPF}(\text{Funcionários})) \text{ — } (\pi_{CPF}(\text{Pacientes}))$	Select CPF From Funcionários except Select CPF From Pacientes
$(\pi_{CPF}(\text{Médicos})) \cap (\pi_{CPF}(\text{Pacientes}))$	Select CPF From Médicos intersect Select CPF From Pacientes

- **Any**

- Permitem outras comparações elemento-conjunto

- Testa se um valor é $>$, $<$, $=$, ... que algum valor em um conjunto

- **SELECT** lista_atributos **FROM** tabela1

- WHERE**

- atributo [$<$ | $>$ | $=$ | $\neq$ | $<>$] **ANY** (conjunto|consulta SQL)

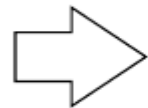
select $a_1, \dots, a_n$

from t_1

where c IN

(select x from t_2

where $d > 5$)



$\pi_{a_1, \dots, a_n} (t_1 \theta X (\pi_x (\sigma_{d > 5} (t_2))))$
 $\theta = t_1.c = t_2.x$

DML - Any



Álgebra	SQL
$\pi_{\text{nome}} \left(\begin{array}{l} \text{Médicos } \theta \text{ X} \\ \theta = \text{Médicos.codm} = \text{Consultas.codm} \\ (\pi_{\text{codm}} (\sigma_{\text{data} = '13/11/2006'} \\ (\text{Consultas}))) \end{array} \right)$	<pre>Select nome From Médicos Where codm = any (ou in) (Select codm From Consultas Where data = '13/11/2006')</pre>
$\pi_{\text{Funcionários.idade}} \left(\begin{array}{l} ((\pi_{\text{idade}} (\text{Funcionários})) \theta \text{ X} \\ \theta = \text{Funcionários.idade} < \text{f2.idade} \\ (\pi_{\text{idade}} (\rho_{\text{f2}} (\text{Funcionários}))) \end{array} \right)$	<pre>Select nome From Funcionários Where idade < any (Select idade From Funcionários)</pre>

- **ALL**

- Permitem outras comparações elemento-conjunto
 - Testa se um valor é $>$, $<$, $=$, ... que TODOS os valores em um conjunto
 - **SELECT** lista_atributos **FROM** tabela1
WHERE
 atributo [$<$ | $>$ | $=$ | $\neq$ | $<>$] **ALL** (conjunto | SQL)
- Não possui um mapeamento para a Álgebra Relacional
 - Não é equivalente a operação de divisão



Exemplo

```
Select nome  
From Funcionários  
Where salário > all  
    (Select salário  
     From Funcionários  
     Where departamento = 'contábil')
```

```
Select nome  
From Funcionários  
Where CPF < > all (or not in)  
    (Select CPF  
     From Pacientes)
```



DML - Subconsulta

- Elemento
 - Permite comparações elemento-valor
 - Testa se um valor é igual ao valor retornado de uma consulta
SELECT lista_atributos **FROM** tabela1
WHERE
 atributo [**>**,**<**,**<>**,**!=**,**=**,**>=**,**<=**] (consulta SQL)
 - Pode ser usado como valor para uma coluna
SELECT lista_atributos, (subconsulta)
[**FROM** TABELA [**WHERE** condicao]]



DML - Subconsulta

```
Select nome  
From Funcionários  
Where salário >  
      (Select salário  
       From Funcionários  
       Where CPF = 22000200002)
```

```
select nome, CPF  
from Médicos  
where CPF < > 10000100001  
and especialidade =  
      (select especialidade  
       from Médicos  
       where CPF = 10000100001)
```

- Otimização
 - Podemos utilizar consultas aninhadas para realizar filtragens prévias nos dados

```
SELECT lista_atributos FROM (SUBCONSULTA)  
WHERE condicao
```

- Ou mesmo para Junções

```
SELECT  
    lista_atributos  
FROM (SUBCONSULTA_A) a  
JOIN (SUBCONSULTA_B) b ON  
    a.pk = b.pk  
[WHERE condicao]
```



Exercícios - IN

- Resolva o que se pede utilizando subconsultas [NOT] IN
 - 1) Nome e CPF dos médicos que também são pacientes do hospital
 - 2) Código e nome dos pacientes com consulta marcada para horários após às 14h
 - 3) Nome e idade dos médicos que possuem consulta com a paciente Ana
 - 4) Número e andar dos ambulatórios onde nenhum médico dá atendimento

Exercícios – ANY e/ou ALL

- Resolva o que se pede utilizando subconsultas ANY e/ou ALL
 - 1) Números e andares de todos os ambulatórios, exceto o de menor capacidade
 - 2) Nome e a idade do médico mais jovem (sem usar a função MIN!)
 - 3) Nome e CPF dos pacientes com consultas marcadas para horários anteriores a todos os horários de consultas marcadas para o dia 14/10/2016
 - 4) Nome e CPF dos médicos que não atendem em ambulatórios com capacidade superior à capacidade de todos os ambulatórios do segundo andar



```
CREATE TABLE Artista
(
    id INTEGER PRIMARY KEY,
    nome VARCHAR(40) NOT NULL,
    CPF CHAR(11) NOT NULL,
    dataNascimento DATE,
    UNIQUE (CPF)
);

CREATE TABLE Evento
(
    id INTEGER PRIMARY KEY,
    descricao VARCHAR(60) NOT NULL,
    numMaxConvidados INTEGER DEFAULT 0,
    CHECK (numMaxConvidados >= 0)
);

CREATE TABLE Atuacao
(
    idArtista INTEGER,
    idEvento INTEGER,
    PRIMARY KEY (idArtista, idEvento),
    FOREIGN KEY (idArtista) REFERENCES Artista,
    FOREIGN KEY (idEvento) REFERENCES Evento(id)
);
```

O sistema também possui uma consulta que integra um de seus relatórios, conforme indicado a seguir.

```
SELECT    A.nome, E.descricao
FROM      Evento E FULL JOIN Atuacao T ON E.id = T.idEvento
          FULL OUTER JOIN Artista A ON T.idArtista = A.id
```





```
CREATE TABLE Artista
```

```
(  
    id INTEGER PRIMARY KEY,  
    nome VARCHAR(40) NOT NULL,  
    CPF CHAR(11) NOT NULL,
```

- a) o nome de todos os artistas combinados com a descrição de todos os eventos.
b) a descrição de todos os eventos e, caso haja artistas alocados, os seus nomes.
c) o nome de todos os artistas e a descrição de todos os eventos em que eles atuam.
d) o nome de todos os artistas e, caso eles participem de eventos, a descrição do evento.
e) o nome de todos os artistas, a descrição de todos os eventos e, caso eles se relacionem, os dois combinados.

```
numMaxConvidados INTEGER DEFAULT 0;  
CHECK (numMaxConvidados >= 0)
```

```
);
```

```
CREATE TABLE Atuacao
```

```
(  
    idArtista INTEGER,  
    idEvento INTEGER,  
    PRIMARY KEY (idArtista, idEvento),  
    FOREIGN KEY (idArtista) REFERENCES Artista,  
    FOREIGN KEY (idEvento) REFERENCES Evento(id)
```

```
);
```

O sistema também possui uma consulta que integra um de seus relatórios, conforme indicado a seguir.

```
SELECT    A.nome, E.descricao  
FROM      Evento E FULL JOIN Atuacao T ON E.id = T.idEvento  
          FULL OUTER JOIN Artista A ON T.idArtista = A.id
```